

Don't Test What You Can Deduce: Causal Discovery with Logical Inference

Jonghwan Kim¹ Sanghack Lee^{1,*}

¹Graduate School of Data Science, Seoul National University, South Korea

*Correspondence: sanghack@snu.ac.kr

uai2026

Deduce high-order CI outcomes from safer lower-order tests, and call the statistical tester only when logic is inconclusive.

1. Why Are We Testing Too Much?

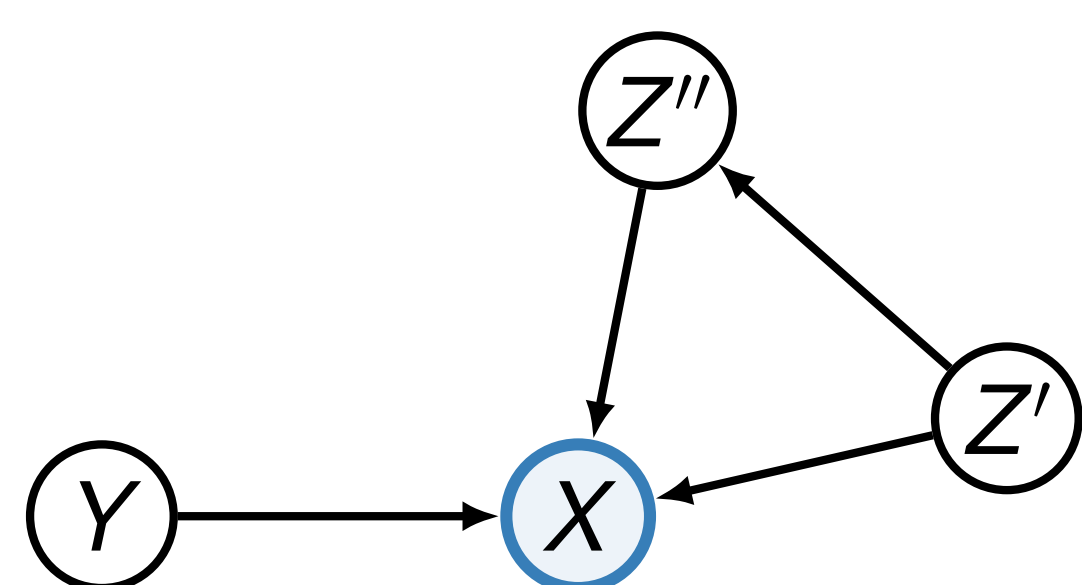
- Constraint-based discovery removes edges using conditional independence tests (CITs):

$$CI(X; Y | \mathbf{S}) \Rightarrow X \perp\!\!\!\perp Y | \mathbf{S} \text{ or } X \not\perp\!\!\!\perp Y | \mathbf{S}.$$

- PC proceeds level-wise, increasing $|\mathbf{S}|$. With finite data, **high-order CITs lose statistical power** and false independencies can trigger fatal **error propagation**.
- Existing deductive checks are often *additive*: test first, then reason. That can improve robustness, but it cannot reduce the baseline test count.

Question: Why estimate a fragile high-order query if its answer is already logically entailed by lower-order outcomes in the PC algorithm's query history?

2. A Motivating Example



At level 0, PC already finds

$$Y \perp\!\!\!\perp Z', \quad Y \perp\!\!\!\perp Z''.$$

It also knows $Y \not\perp\!\!\!\perp X$ and $X \not\perp\!\!\!\perp Z'$.

Therefore the level-1 query

$$CI(X; Y | Z')$$

is **logically dependent**: no new sample-based test is needed.

Key resource: the query history. When a PC(-like) algorithm reaches level k , it has already completed every lower-order level and we can reuse many dependence and independence outcomes.

3. DEDUCE: The Logical Gatekeeper

Let $\mathbf{S} = \mathbf{S}' \sqcup \{Z\}$. DEDUCE recursively applies sound rules derived from graphoid axioms and faithfulness.

Deduce dependence

$$\begin{aligned} X \not\perp\!\!\!\perp Y | \mathbf{S}', \quad X \perp\!\!\!\perp Z | \mathbf{S}' \\ \Rightarrow X \not\perp\!\!\!\perp Y | \mathbf{S}, \\ X \perp\!\!\!\perp Y | \mathbf{S}', \quad X \not\perp\!\!\!\perp Z | \mathbf{S}', \\ Y \not\perp\!\!\!\perp Z | \mathbf{S}' \\ \Rightarrow X \not\perp\!\!\!\perp Y | \mathbf{S}. \end{aligned}$$

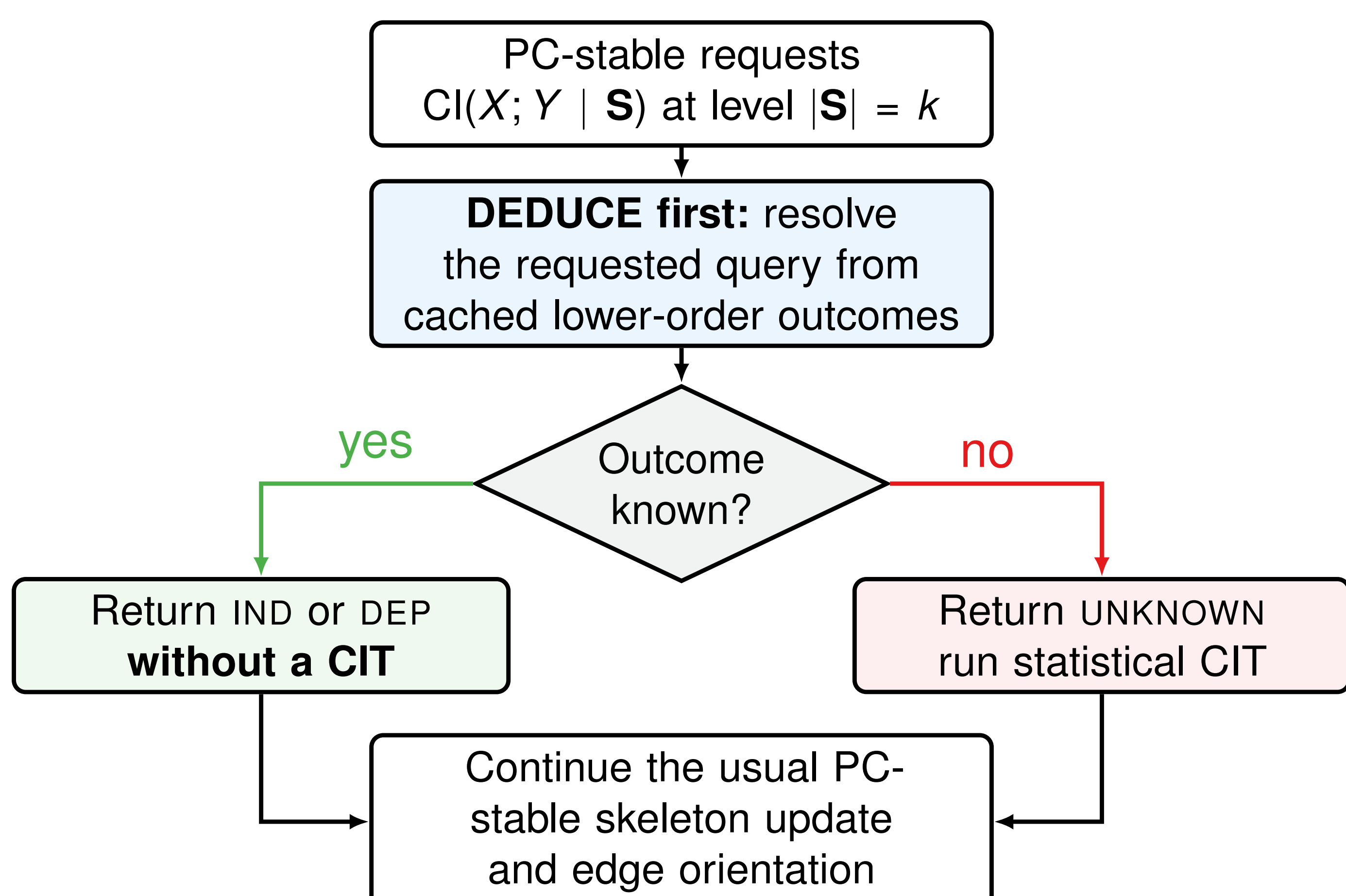
The first rule also holds with $Y \perp\!\!\!\perp Z | \mathbf{S}'$ by symmetry.

Deduce independence

$$\begin{aligned} X \perp\!\!\!\perp Y | \mathbf{S}', \\ X \perp\!\!\!\perp Z | \mathbf{S}' \text{ or } Y \perp\!\!\!\perp Z | \mathbf{S}' \\ \Rightarrow X \perp\!\!\!\perp Y | \mathbf{S}. \end{aligned}$$

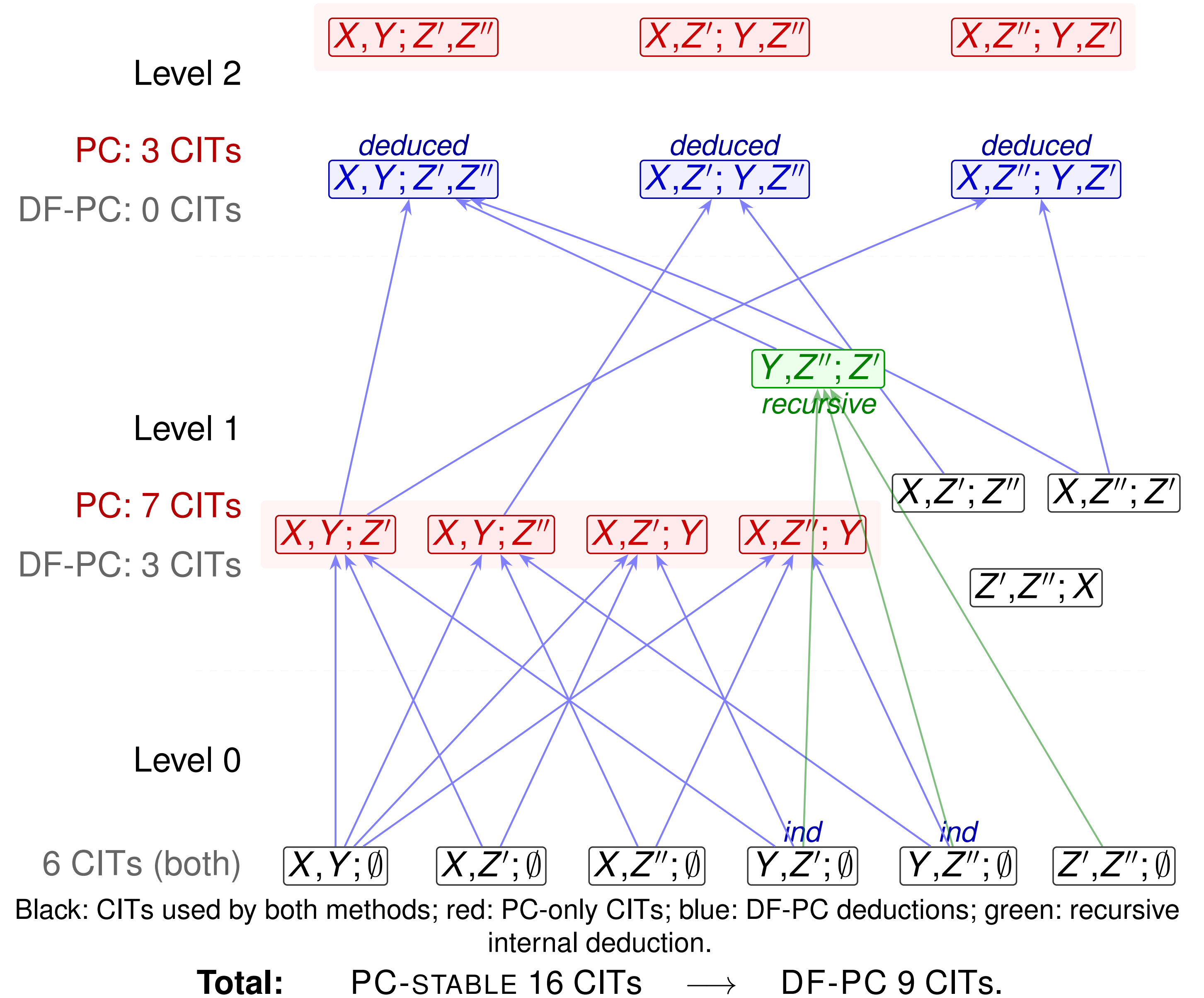
A lower-order independence remains a reusable premise even after its edge was removed.

4. DF-PC: Deduce First, Test Only as Fallback



- No change to the outer PC-stable schedule:** same edge-removal and orientation logic.
- Only the query mechanism changes:** DEDUCE($X, Y, \mathbf{S}$) intercepts every requested CIT.

5. What Gets Skipped during Structure Learning?



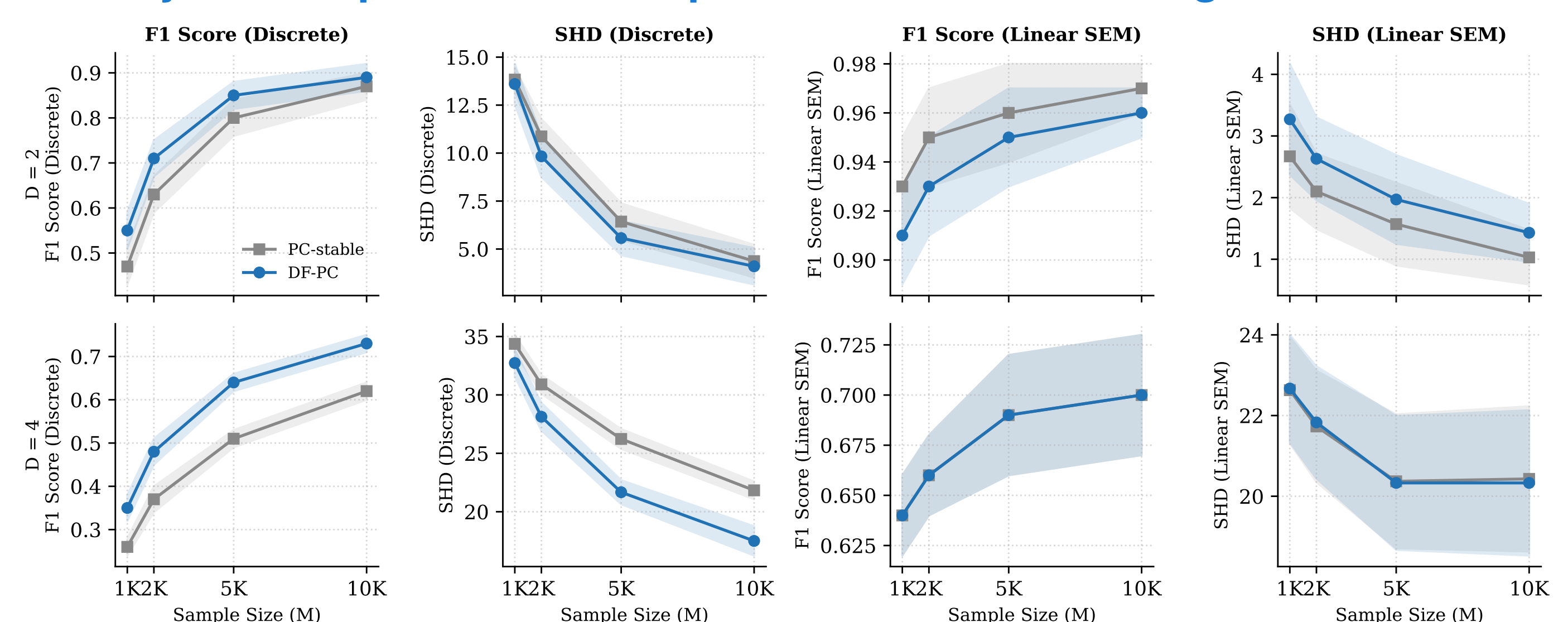
6. Theoretical Guarantees

- ① **Soundness of DEDUCE:** with a faithful Bayesian network and oracle tester, DEDUCE returns the correct CI outcome.
- ② **Soundness and completeness of DF-PC:** under oracle tester, DF-PC recovers the same pattern as PC-stable.
- ③ **Test complexity:** worst case: $O(m^{n+1})$ best case: $\Theta(m^2)$.

The best case is tight: recovering some non-empty graphs (e.g., collider hub) require only the initial pairwise marginal tests.

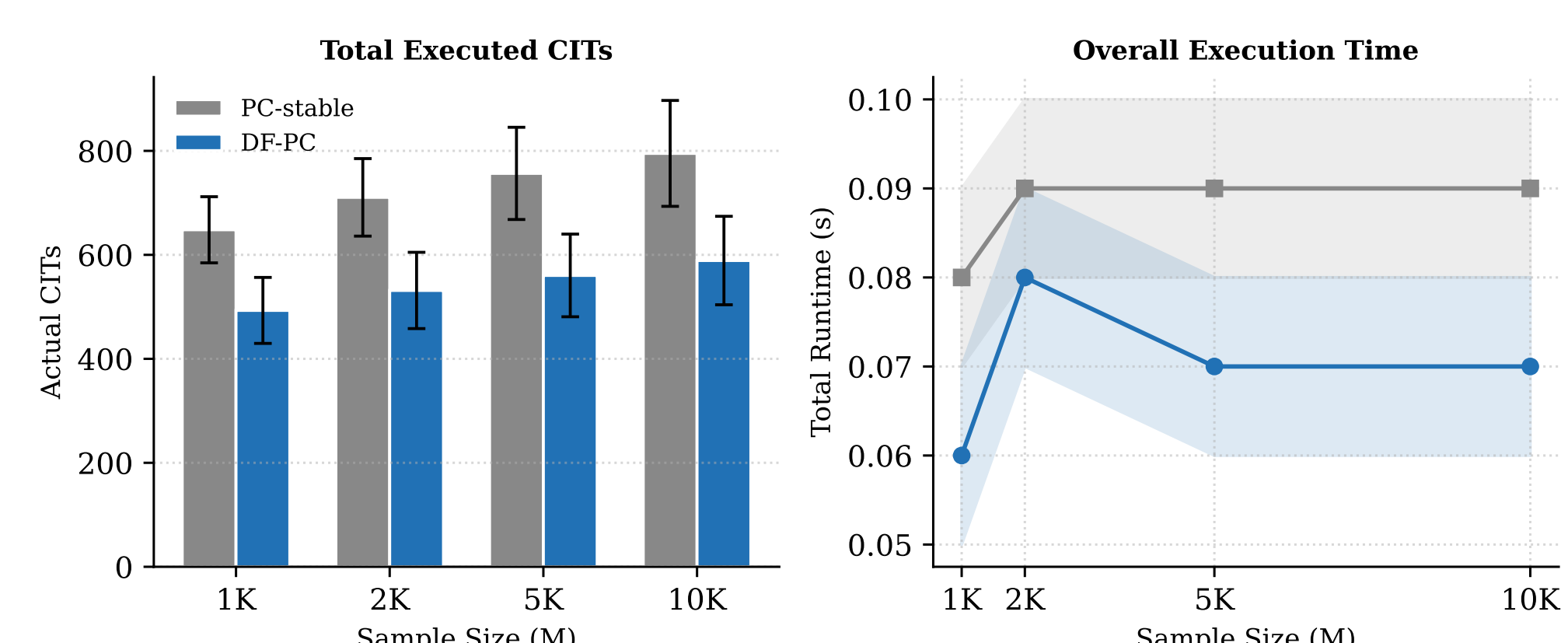
7. Empirical Results

Accuracy: DF-PC preserves or improves structure learning



Takeaway. DF-PC is consistently competitive and often better than PC-stable, with the clearest gains in **discrete data** and at **higher density**.

Efficiency: DF-PC executes fewer CITs and runs faster



Takeaway. By skipping many higher-order tests, DF-PC reduces the number of executed CITs and improves runtime.

8. Conclusion

- We present DF-PC, a variant of PC that not only uses independence findings as edge-deletion signals but also as ingredients of logical deduction.
- This approach effectively reduces higher-order CITs, lowering computational cost while preserving or improving learning accuracy.